

PID Motor Speed Control

I. Experiment Objective

Learn and master how to use the incremental PID algorithm on the ESP32-S3 platform to perform closed-loop speed control of a DC gear motor. Through encoder feedback of the actual speed, the PID controller dynamically adjusts the PWM duty cycle to stabilize the actual motor speed near the target value. Optimization strategies such as speed feedforward, start-up boost, disturbance recovery, and high-speed compensation are also introduced to improve the motor's response performance across the full speed range.

II. Experiment Principle

1. Keywords

Keyword	Explanation
PID controller	Proportional-Integral-Derivative controller. It calculates the error between the target value and the actual value, and outputs a control quantity to stabilize the system at the target value. This experiment uses incremental PID, which outputs the change in PWM each time rather than the absolute value.
Kp / Ki / Kd	The proportional coefficient (P) determines the strength of responding to the current error; the integral coefficient (I) eliminates steady-state error; the derivative coefficient (D) predicts the trend of error change. In this experiment Kp=1.15, Ki=0.24, Kd=0.0 (disabled).
Encoder feedback	The DC gear motor has a built-in Hall encoder that outputs 1320 pulses per revolution. The PCNT peripheral counts pulse increments, which are converted to the actual linear velocity (m/s), forming a closed-loop feedback path.
Speed feedforward	Directly calculates the base PWM value (feedforward gain 120.0) from the target speed and adds it to the PID output. Under high-speed, high-load conditions it reduces the PID burden and steady-state error.
Dead-zone compensation	The motor PWM drive has a dead zone (200 ticks). The control value must add the dead-zone offset before output to ensure the motor can still start at small duty cycles.

2. Technical Architecture

The program adopts a layered modular design with three layers from top to bottom: motor layer (PID control logic) → encoder layer (encoder pulse counting) → pwm_motor layer (PWM motor drive), while reusing the official `pid_ctr1` and `bdc_motor` components.

```

main/main.c
└─ Motor_Init() / Motor_Set_Speed() / Motor_Get_Speed()
    └─ components/motor/          PID control task + optimization strategies
        └─ components/encoder/    PCNT encoder pulse counting
            └─ components/pwm_motor/ MCPWM DC motor drive
                └─ espressif_pid_ctrl    official PID control library
                    └─ espressif_bdc_motor    official DC motor drive library

```

The PID control task runs `Motor_PID_Ctrl()` every 10ms: read the encoder → calculate pulse increments → low-pass filter → convert to linear velocity → incremental PID calculation → add feedforward/boost/recovery/compensation → PWM output with limiting.

III. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable (Type-C)	✓ Required
Motor (with encoder)	✓ Required (built into the chassis)
Battery (12V)	✓ Required (chassis power supply)

2. Prerequisites

1. Set up the ESP32-IDF development environment
2. Connect the MicroROS-V2 control board to the computer via USB
3. The chassis motor power supply is normal (insert the battery and turn on the chassis power switch)
4. Make sure the motor wheels are lifted or the car is suspended to avoid unexpected movement

3. Compile and Flash

```

cd ~/esp/base_samples/pid_speed
source ~/esp/esp-idf/export.sh
idf.py build flash monitor

```

You can also run the steps separately:

```
cd ~/esp/base_samples/pid_speed
source ~/esp/esp-idf/export.sh
idf.py build
idf.py flash
idf.py monitor
```

To exit the monitor: press `Ctrl + ]`.

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After flashing and running the program, you can see the following output in the serial monitor terminal:

```
hello yahboom
I (1002) MAIN: Nice to meet you!
I (2012) MOTOR: Start Motor_Task with core:1
I (2012) MAIN: Motor_Set_Speed!
I (2112) MAIN: Read_Speed: 0.283 m/s, 0.275 m/s
I (2212) MAIN: Read_Speed: 0.521 m/s, 0.510 m/s
I (2312) MAIN: Read_Speed: 0.748 m/s, 0.735 m/s
I (2412) MAIN: Read_Speed: 0.896 m/s, 0.882 m/s
I (2512) MAIN: Read_Speed: 0.965 m/s, 0.951 m/s
I (2612) MAIN: Read_Speed: 0.992 m/s, 0.978 m/s
...
```

The speeds of the two wheels gradually accelerate from 0 to around 1.0 m/s. After PID adjustment, they stabilize near the target speed with small fluctuations. About 10 seconds later, the motor stops.

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/pid_speed/main/main.c`
- PID control: `~/esp/base_samples/pid_speed/components/motor/motor.h`, `motor.c`
- Encoder driver: `~/esp/base_samples/pid_speed/components/encoder/encoder.h`, `encoder.c`
- PWM motor driver: `~/esp/base_samples/pid_speed/components/pwm_motor/pwm_motor.h`, `pwm_motor.c`

1. Directory Structure

```
pid_speed/
├── CMakeLists.txt
├── main/
│   ├── CMakeLists.txt
│   └── main.c
```

```

├─ components/
│   └─ motor/           # Motor PID control logic
│       └─ motor.c
│       └─ motor.h
│   └─ encoder/        # Encoder pulse counting
│       └─ encoder.c
│       └─ encoder.h
│   └─ pwm_motor/      # PWM motor drive layer
│       └─ pwm_motor.c
│       └─ pwm_motor.h
└─ managed_components/
    └─ espressif__pid_ctrl/ # Official PID control library
    └─ espressif__bdc_motor/ # Official DC motor drive library

```

2. Main Program Logic

`main/main.c` implements a simple open-loop to closed-loop transition demonstration:

1. Delay 1 second to wait for the system to stabilize
2. Call `Motor_Init()` to initialize the encoder, PWM driver, and PID control task
3. Set the target speed to `1.0 m/s` (the same for both left and right wheels)
4. Loop to read and print the actual speeds of both wheels
5. After running 100 cycles (about 10 seconds), call `Motor_Stop(STOP_COAST)` to stop the motor (coast to stop)

Key code:

```

Motor_Init();
Motor_Set_Speed(1.0f, 1.0f); // Set target speed 1.0 m/s

while (1) {
    Motor_Get_Speed(&speed_m1, &speed_m2);
    ESP_LOGI(TAG, "Read_Speed: %.3f m/s, %.3f m/s", speed_m1, speed_m2);
    vTaskDelay(pdMS_TO_TICKS(100));
    count++;
    if (count >= 100) {
        Motor_Stop(STOP_COAST);
    }
}

```

3. Key Components

motor layer (PID control logic)

Files: `components/motor/motor.h`, `components/motor/motor.c`

Physical parameters:

Parameter definition file: `~/esp/base_samples/pid_speed/components/motor/motor.h`

Macro	Value	Description
<code>WHEEL_DIAMETER</code>	67.5f	Wheel diameter (unit mm)
<code>MOTOR_ENCODER_CIRCLE</code>	1320.0f	Encoder pulses per motor revolution
<code>MOTOR_PID_PERIOD</code>	10	PID control period (unit ms)
<code>MOTOR_MAX_SPEED</code>	1.0f	Maximum speed limit (unit m/s)
<code>MOTOR_ENCODER_SIGN_M1</code>	-1	M1 encoder direction sign
<code>MOTOR_ENCODER_SIGN_M2</code>	+1	M2 encoder direction sign

PID parameters:

Macro	Default Value	Description
<code>MOTOR_PID_DEFAULT_KP</code>	1.15f	Proportional coefficient
<code>MOTOR_PID_DEFAULT_KI</code>	0.24f	Integral coefficient
<code>MOTOR_PID_DEFAULT_KD</code>	0.0f	Derivative coefficient (currently disabled)
<code>MOTOR_SPEED_FILTER_ALPHA</code>	0.35f	Speed first-order low-pass filter coefficient

Optimization strategy parameters:

- **Start-up boost:** After the target speed is given, if the actual speed is lower than `MOTOR_START_SPEED_TH` (0.03 m/s), directly output a fixed PWM value (26 for both M1 and M2) for 12 cycles (120ms) to help the motor overcome static friction and start.
- **Disturbance recovery:** When the speed error exceeds `MOTOR_RECOVER_ERR_TH` (0.035 m/s), add an extra compensation PWM value of 10 to speed up the recovery to the target speed.
- **Speed feedforward:** `MOTOR_M1_FEEDFORWARD_GAIN = 120.0f`, calculates the feedforward PWM based on the target speed to reduce the steady-state error of the PI controller under high-speed, high-load conditions.
- **High-speed compensation:** When the target speed exceeds 0.70 m/s and the PWM output exceeds 120, add an extra compensation PWM value of 16 to improve high-speed load performance.

Core functions:

- `Motor_Init()` — Initializes the encoder and PWM driver, and creates the PID control task (pinned to core 1, stack size 10KB, priority 10)
- `Motor_Set_Speed(float speed_m1, float speed_m2)` — Sets the target speeds of both wheels (range -1.0 ~ 1.0 m/s), automatically limited when out of range
- `Motor_Get_Speed(float *speed_m1, float *speed_m2)` — Gets the current actual speeds
- `Motor_Stop(bool brake)` — Stops the motor; `STOP_COAST` coasts to stop, `STOP_BRAKE` brakes
- `Motor_Update_PID_Parm(float pid_p, float pid_i, float pid_d)` — Updates PID parameters at runtime

- `Motor_Read_PID_Parm(float *out_p, float *out_i, float *out_d)` — Reads the current PID parameters

PID calculation flow:

The PID control task (`Motor_Task`) runs `Motor_PID_Ctrl()` every 10ms. The execution flow is as follows:

1. Read the current encoder count
2. Calculate the pulse increment in the period (with direction sign)
3. Apply a first-order low-pass filter to smooth the pulse value
4. Convert the pulse value to the actual linear velocity (m/s)
5. Calculate the error between the target pulses and the actual pulses
6. Call `pid_compute()` to perform the incremental PID calculation
7. Add speed feedforward, start-up boost, disturbance recovery, and high-speed compensation
8. Limit the PWM value and output it to the motor driver

The PID controller is configured for incremental calculation (`PID_CAL_TYPE_INCREMENTAL`), with integral limiting of ± 1000 and output limiting of ± 200 .

encoder layer (encoder driver)

Files: `components/encoder/encoder.h`, `components/encoder/encoder.c`

Uses the PCNT (pulse counter) peripheral of the ESP32-S3 to decode the encoder signals.

GPIO pin assignment:

Encoder	Phase A	Phase B
M1 (left wheel)	GPIO5	GPIO4
M2 (right wheel)	GPIO7	GPIO6

PCNT configuration:

- Count upper limit: 1000, lower limit: -1000
- Accumulate counting enabled (`accum_count = true`)
- Glitch filter: 1000ns
- Add high/low limit watchpoints to handle overflow

Dual-channel configuration implements quadrature decoding: channel A detects edge signals and identifies the level direction, channel B is configured the same way, and the A/B phase combination determines the rotation direction.

Core functions:

- `Encoder_Init()` — Initializes the M1 and M2 encoders
- `Encoder_Get_Count(uint8_t encoder_id)` — Gets the current accumulated pulse count of the specified encoder (`ENCODER_ID_M1 = 1`, `ENCODER_ID_M2 = 2`)

pwm_motor layer (motor driver)

Files: `components/pwm_motor/pwm_motor.h`, `components/pwm_motor/pwm_motor.c`

Uses the MCPWM peripheral and the `bdc_motor` component to drive the DC motors.

GPIO pin assignment:

Motor	Channel A	Channel B
M1 (left wheel)	GPIO13	GPIO14
M2 (right wheel)	GPIO11	GPIO12

Note: In the `bdc_motor` initialization, the `pwma` and `pwm_b` pin parameters are swapped relative to the motor A/B channels. The actual wiring is:

- M1: `pwma = GPIO14 (M1B)`, `pwm_b = GPIO13 (M1A)`
- M2: `pwma = GPIO12 (M2B)`, `pwm_b = GPIO11 (M2A)`

PWM configuration:

- Timer resolution: 10 MHz (1 tick = 0.1us)
- PWM frequency: 25 KHz
- Maximum duty cycle: 400 ticks (=10MHz/25KHz)
- Dead-zone compensation: 200 ticks
- Actual maximum control value: 200 ticks (=400-200)

Logic direction:

- M1 positive direction: +1 (default)
- M2 positive direction: -1 (mirror mapping, ensuring both wheels move in the same direction as the positive direction)

Core functions:

- `PwmMotor_Init()` — Initializes the MCPWM devices for M1 and M2
- `PwmMotor_Set_Speed(motor_id_t motor_id, int speed)` — Sets the PWM value of the specified motor (with dead-zone compensation and logic direction handling)
- `PwmMotor_Stop(motor_id_t motor_id, bool brake)` — Stops the motor (coast/brake)

VI. Common Problems

Problem	Cause	Solution
Motor does not turn	Battery not powered or power switch not turned on	Check the battery connection and the chassis power switch

Problem	Cause	Solution
Motor only turns in one direction or only one wheel turns	Encoder direction sign misconfigured	Adjust the signs of <code>MOTOR_ENCODER_SIGN_M1</code> / <code>MOTOR_ENCODER_SIGN_M2</code>
Large speed fluctuation	Inappropriate PID parameters or encoder signal interference	Adjust the <code>MOTOR_PID_DEFAULT_KP</code> / <code>KI</code> parameters and check the encoder wiring
Motor hums but does not turn	Insufficient dead-zone compensation	Adjust the <code>PWM_MOTOR_DEAD_ZONE</code> parameter
Speed cannot reach the target at high speed	Low battery voltage or insufficient feedforward compensation	Check the battery level and adjust <code>MOTOR_M1_FEEDFORWARD_GAIN</code>